DAY 01 · THEORY HANDOUT

AI, ML and DL: First Steps

What the names mean, how learning works, and where we will run our notebooks.

KGP Talkie

Scan to subscribe

Laxmi Kant Tiwari · kgptalkie.com

AI, ML, and DL: three linked names

Artificial Intelligence is the broad field. ML is inside AI. DL is inside ML.



A fixed set of rules can help a computer choose an answer. **Machine learning** instead learns useful patterns from examples.

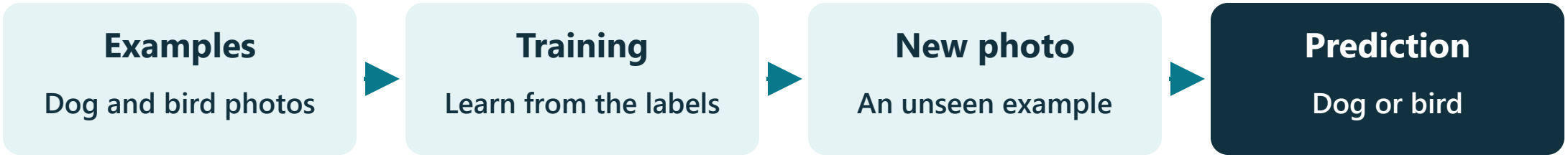
Deep learning uses neural networks with many connected layers. A layer is one small processing step. "Deep" refers to the layers.

> These are related names, not three separate kinds of computer.

How does a model learn?

Data means the examples we give it. A **label** is the known answer for one example.

A label is the known answer: dog or bird.



We check new photos to see whether the learned model works.

Training is the learning stage. The program changes its internal numbers to find patterns that help match photos to their labels.

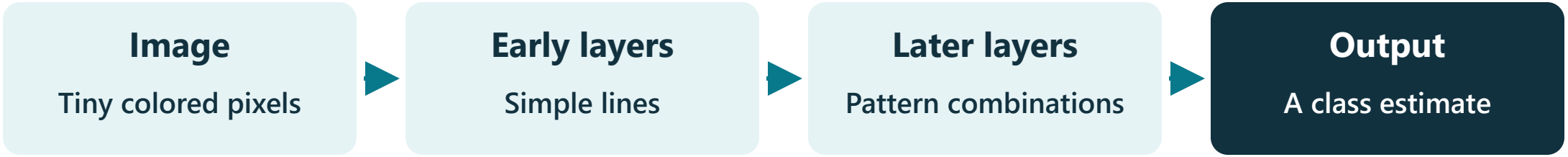
A **model** is the system with those learned numbers. A **prediction** is its answer for a new example. It can be wrong.

> Learning from known answers is called supervised learning. Testing uses examples that were not used for learning.

What makes learning deep?

A **neural network** is a model made of connected processing layers.

Neural network = connected layers of small calculations



Early layers can find simple patterns. Later layers can combine them. The model learns which patterns help with the task.

More layers can help with hard patterns, but they often need more data and computer work. They do not give a computer human thought.

> DL is one kind of ML. A small ML model can still be a good choice for a simple task.

GenAI, prompts, and language models

Generative AI makes new content, such as text, pictures, or sound.

Prompt

The instruction we give:
"Explain a rainbow."

LLM

Large Language Model:
a model trained on text.

AI agent

Uses a model and tools
to carry out task steps.

An LLM creates text one small piece at a time. A piece may be a word or part of a word. It learns patterns from many text examples.

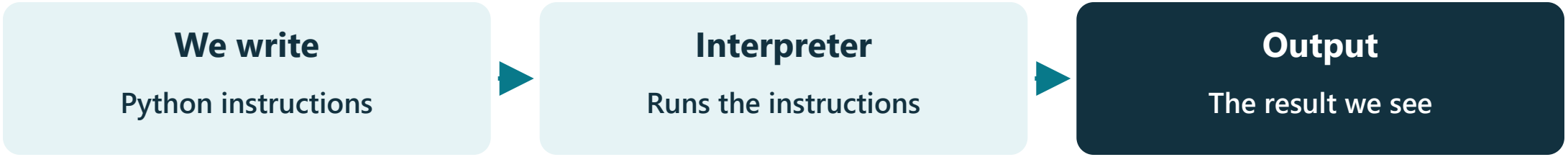
An agent can use tools, such as search or a calculator. We set what it may do and check its results.

> A clear reply can still contain a false fact. Check important answers before using them.

Python: our way to give instructions

Python is a programming language. A **program** is a set of instructions for a computer.

Interpreter = software that runs Python



Python is easy to read and has many ready tools for data and AI. A **library** is a collection of code we can reuse.

Python is one choice. C++ often runs fast parts behind the scenes. JavaScript can build web pages. We start with Python to learn the ideas with less code.

> Installing Python gives us the interpreter. We also need a place to write and run our work.

IDE, Jupyter, and kernel

An **IDE** is an Integrated Development Environment: an app with tools for writing and running code.

IDE: the workspace

Example: VS Code.

Write and manage files.

Jupyter: the notebook

Text + code + results.

A file ending in .ipynb.

Kernel: the runner

Runs the code cells.

Remembers their values.

A **Jupyter Notebook** is a document with separate cells. A Markdown cell holds text or pictures. A code cell holds instructions to run.

The **kernel** is the running Python process behind the notebook. The browser shows the page; the kernel does the calculations.

> We can use Jupyter in a browser for this course. Installing VS Code is optional.

Setup 1: install Python on Windows

The course environment uses **Python 3.12**, so we use the same version for these notebooks.

Download and install

Install Python 3.12 with the Windows installer. Choose Windows installer (64-bit) for a 64-bit Intel/AMD PC. Run it. Keep the Python launcher selected. Select Add python.exe to PATH, then Install Now.

Check the installation

Open PowerShell from the Windows Start menu.

```
py -3.12 --version
```

py chooses an installed Python. **-3.12** asks for this version. The result should begin with Python 3.12.

PATH is the list of folders Windows searches for programs. The installer can add Python to that list.

> These setup pages are for Windows. The AI terms and notebook lessons are the same on other systems.

Setup 2: open the course folder

Keep the notebooks, requirements file, and launcher together in one course folder.

Get the files

Open the course repository. Choose **Code > Download ZIP**.
Right-click the ZIP, choose **Extract All**, then open the extracted folder that contains README.md and requirements.txt.

Open PowerShell here

Click the address bar of this folder in File Explorer. Type **powershell** and press Enter.

```
Get-ChildItem
```

This lists files. Look for **requirements.txt** and **materials** before the next step.

A **repository** is the shared project folder on GitHub. README.md explains how to use it.

requirements.txt lists the Python packages this course needs. Running setup from this folder lets Python find that list.

> If the folder is already on your computer, use that copy. There is no need to download it again.

Setup 3: make the course environment

An **environment** is a separate place for this project's Python and installed libraries.

Create it

```
py -3.12 -m venv .venv
```

-m runs a Python tool. **venv** makes the environment. **.venv** is its folder name.

Install its libraries

```
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
```

pip installs packages. **-r** tells it to read the list. Using this Python installs them inside the course environment.

The same environment is used for the whole course. Separate environments let different projects use different package versions.

This step needs the internet. Some packages, including TensorFlow, are large. Wait until PowerShell shows its prompt again.

> On this prepared computer, .venv already exists. On a new computer, make it once and install the listed packages.

Setup 4: open and run a notebook

Register the course kernel once, then use the launcher to open Jupyter.

Register the runner

```
.\.venv\Scripts\python.exe -m ipykernel install
--prefix .venv --name kgp-talkie --display-name
"KGP Talkie (course)"
```

ipykernel connects Python to Jupyter. The display name is the kernel we will select.

Open the notebook

Double-click **start-notebooks.cmd**. A browser opens Jupyter. Open a file inside **materials/day-02/notebooks/**. Choose **KGP Talkie (course)** if asked. Select a code cell. **Shift + Enter** runs it and moves to the next cell.

The launcher uses .venv directly. Leave its terminal window open while working. Close Jupyter's running server when you finish.

Run cells from top to bottom. Restarting the kernel clears remembered values, so run the cells again after a restart.

> We are ready when Jupyter opens the notebook, the course kernel starts, and a code cell shows its output.

DAY 01 · COMPLETE

Thank You

Follow KGP Talkie for new lessons, course updates and live sessions.

Scan to follow

Instagram
@kgptalkie

WhatsApp Channel
KGP Talkie